



Unidade 1 — Back-End e Front-End



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Compreender** o conceito de comunicação assíncrona;
- **Explicar** o que é AJAX e por que ele revolucionou a web;
- **Diferenciar** XMLHttpRequest e Fetch API;
- **Implementar** requisições assíncronas com `fetch()` ;
- **Analisar** o fluxo de dados em uma aplicação assíncrona.

Taxonomia de Bloom: Compreender, Explicar, Diferenciar, Implementar, Analisar.



O Problema: Síncrono vs. Assíncrono

Comunicação síncrona (antes de AJAX):

Usuário clica → Página recarrega → Servidor processa → Nova página carrega

- A página **trava** durante o processamento;
- Experiência **lenta** e **frustrante**;
- **Desperdício** de banda (recarrega tudo).

Comunicação assíncrona (com AJAX):

Usuário clica → Requisição em segundo plano → Servidor processa → Apenas parte da página atualiza

- A página **não trava**;
- Experiência **fluida** e **responsiva**;
- **Eficiente** (atualiza apenas o necessário).



O que é AJAX?

- **AJAX** = Asynchronous JavaScript And XML;
- **Não é uma tecnologia** — é uma **técnica** que combina:
 - JavaScript (programação);
 - XMLHttpRequest ou Fetch API (comunicação);
 - DOM (manipulação da interface);
 - JSON (formato de dados — antes era XML).

Por que existe? Para permitir que o navegador se comunique com o servidor **sem recarregar a página**.



XMLHttpRequest (Legado)

```
// Criar o objeto XMLHttpRequest
const xhr = new XMLHttpRequest();

// Configurar a requisição (método, URL, assíncrono)
xhr.open('GET', '/api/usuarios', true);

// Definir o callback para quando a resposta chegar
xhr.onreadystatechange = function () {
  if (xhr.readyState === 4 && xhr.status === 200) {
    const usuarios = JSON.parse(xhr.responseText);
    console.log(usuarios);
  }
};

// Enviar a requisição
xhr.send();
```

- Foi a **primeira** forma de fazer AJAX;
- Ainda funciona, mas é **verbosa** e **complexa**;



Fetch API (Moderno)

```
// Requisição GET simples
fetch('/api/usuarios')
  .then(response => response.json())
  .then(usuarios => {
    console.log(usuarios);
  })
  .catch(error => {
    console.error('Erro:', error);
  });
```

- API **moderna** e **nativa** do navegador;
- Baseada em **Promises**;
- Sintaxe **mais limpa** e **legível**;
- Suporta **async/await**.



Fetch com Async/Await

```
// Função assíncrona com async/await
async function carregarUsuarios() {
  try {
    const response = await fetch('/api/usuarios');
    const usuarios = await response.json();
    console.log(usuarios);
  } catch (error) {
    console.error('Erro:', error);
  }
}

carregarUsuarios();
```

- `async` declara que a função é assíncrona;
- `await` pausa a execução até a Promise ser resolvida;
- `try/catch` trata erros de forma mais intuitiva.



Requisição POST com Fetch

```
async function criarUsuario(nome, email) {
  try {
    const response = await fetch('/api/usuarios', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ nome, email })
    });

    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }

    const usuario = await response.json();
    console.log('Usuário criado:', usuario);
  } catch (error) {
    console.error('Erro ao criar usuário:', error);
  }
}
```




Fluxo Completo de uma Aplicação Assíncrona

FRONT-END

1. Usuário interage (clique, formulário)
2. JavaScript captura o evento
3. `fetch()` envia requisição HTTP
4. Página continua responsiva
5. Resposta chega (callback/await)
6. DOM é atualizado com os novos dados

↓ HTTP (JSON)

BACK-END

7. Servidor recebe a requisição
8. Processa a lógica de negócio
9. Consulta o banco de dados
10. Retorna resposta JSON



Erros Comuns

Erro	Consequência	Solução
Não tratar erros do fetch	Aplicação "quebra" silenciosamente	Usar try/catch ou .catch()
Não verificar response.ok	Trata erro como sucesso	Verificar antes de processar
Esquecer o Content-Type no POST	Servidor não interpreta o body	Definir headers corretamente
Usar fetch síncrono	Página trava	Sempre usar async/await
Não atualizar o DOM após sucesso	Interface desatualizada	Atualizar no .then() ou após await



Síntese da Aula

Conceitos principais:

1. **AJAX** é a técnica de comunicação assíncrona entre navegador e servidor;
2. A **Fetch API** é a forma moderna de fazer requisições HTTP no navegador;
3. **async/await** simplifica o código assíncrono, tornando-o mais legível;
4. O **DOM** é atualizado dinamicamente com os dados recebidos do servidor;
5. O **tratamento de erros** é essencial em qualquer requisição assíncrona.

Pergunta reflexiva:

Se AJAX não existisse, como seria a experiência do usuário no Gmail ou no Facebook?

Próxima aula: Boas práticas em Front-End e Back-End — Vamos consolidar as práticas que todo desenvolvedor deve adotar.

